IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

# Yenepoya Institute of Arts Science Commerce and Management

## Low Level Design

## on

## Identity Security Framework for Energy

**Student Name: Shalaya P B**

**Industry Mentor:**
**Shashank Koram**

**Guided by:Suzana Sharol Prabhakar**

# Table of Contents

1. Introduction. ....3
   1.1. Scope of the document. .... 3
   1.2. Intended Audience.... 3
   1.3. System overview.... 4

2. System Design. ....4
   2.1. Application Design....4
   2.2. Process Flow.... 4
   2.3. Information Flow....5
   2.4. Components Design ....5
   2.5. Key Design Considerations ....5
   2.6. API Catalogue. .... 6

3. Data Design....6
   3.1. Data Model.... 6
   3.2. Data Access Mechanism....6
   3.3. Data Retention Policies .... 6
   3.4. Data Migration ....7

4. Interfaces....7

5. State and Session Management....7

6. Caching .... 7

7. Non-Functional Requirements .... 8
   7.1. Security Aspects....8
   7.2. Performance Aspects....8

8. References.... 8

# 1. Introduction

This document outlines the granular, low-level architecture for the Energy Monitoring web application. It breaks down the specific software components, database models, and function-level logic necessary to build and deploy the system. By explicitly defining the server routing behaviors, data-generation algorithms, and security mechanisms, this document serves as the definitive technical guide for developers working on the application's implementation.

## 1.1 Scope of the Document

- Data Models & Persistence: A detailed breakdown of the SQLite database logic, specifically focusing on the fields and constraints mapped within the User class.
- Routing & Controllers: Specifications for all Flask application endpoints, clarifying the flow between public pages, authentication gateways, and the restricted dashboard.
- Algorithm Logic: A deep dive into the custom deterministic functions used to dynamically simulate power usage, solar generation, and smart-device warnings on a daily cycle.
- Security Implementation: Technical definitions covering how Werkzeug hashes passwords before storage and how Flask-Login manages persistent session cookies.
- Access Control Workflows: Explanations of the conditional logic that restricts certain network requests—such as deleting an account—to the platform administrator.

## 1.2 Intended Audience

- Software Engineers: Those responsible for writing the backend Python logic, managing the Flask environment, and binding variables to the HTML frontend.
- Quality Assurance (QA) Teams: To help construct targeted testing strategies that probe for vulnerabilities in the routing barriers or data-generation logic.

### 1.3 System Overview

The application acts as a secure, centralized reporting hub where individuals can log in to track simulated electrical statistics. The frontend dynamically renders a dashboard highlighting daily kWh usage, grid conditions, and active facility statuses (such as HVACs or Data Centres). From a technical standpoint, the platform strictly isolates this data—ensuring that users can only access their personal dashboard post-authentication. Concurrently, it features a backend administrative layer that permits system moderators to view the entire user registry and cleanly remove compromised or obsolete accounts from the database.

## 2. System Design

### 2.1 Application Design

- Controller(app.py): Handles Flask routing logic, intercepts requests with @login_required, and maps backend data to frontend templates.
- Model (SQLAlchemy): Functions as the Object-Relational Mapper (ORM), binding Python class objects directly to the underlying energyguard.db SQLite file.

### 2.2 Process Flow

➢ Registration Workflow

- Intake: Parses POST arrays (Username, Email, Password) from /signup.
- Validation: Filters for unique DB constraints and logical string lengths.
- Hashing: Processes raw strings via Werkzeug's generate_password_hash.
- Commit: Executes db.session.add() and redirects seamlessly to the login gateway.

➢ Authentication Workflow

- Intake: Scans for user identifier (Username OR Email) and password parameters.
- Query Match: ORM executes. filter_by() searching for valid records.

- Verification: Evaluates security string natively via check_password_hash.
- Session Mount: Triggers Flask-Login's login_user() to issue encrypted session cookies immediately.

## 2.3 Information Flow

- Trigger: Client browser executes HTTP GET/POST routines.
- Evaluation: Flask intercepts securely, validates secret-key cookies, and calls internal functions.
- Data Extraction: The backend either fires ORM queries to SQLite or computes numeric generation via get_energy_data().
- Hydration: Raw variable dictionaries are passed into Jinja templates, compiling standard HTML for final transmission.

## 2.4 Component Design

- Auth Module: Specifically integrates the UserMixin framework and @login_manager loaders to automate session lifecycles.
- Simulation Engine: Contains the procedural get_energy_data(user_id) algorithms responsible for generating hardware statistics based on time-encoded seeds.
- Database Framework: Employs Flask-SQLAlchemy to structure schemas and handle programmatic row modifications.
- Presentation Layer: Jinja-enabled HTML files (/templates) coupled purely with local vanilla CSS (style.css).

## 2.5 Key Design Considerations

- ORM Adoption: Relying completely on SQLAlchemy prevents direct raw-SQL evaluation, systematically blocking standard SQL-Injection paths.

- Deterministic Variables: Mock analytics rely on localized seeds (User ID + Current Date format), guaranteeing identical dashboard values on rapid page refreshes.
- Hybrid Login Fields: Backend query logic programmatically attempts to filter identities utilizing either the email column or the username column interchangeably.

### 2.6 API Catalogue

- / [GET]: Serves the default public landing layout.
- /signup [GET, POST]: Parses strict parameter restrictions and commits records.
- /login [GET, POST]: Validates Werkzeug hashes and generates state cookies.
- /logout [GET]: Executes native session destruction routines.
- /dashboard [GET]: Executes simulation processing scripts and mounts protected visuals.
- /admin/delete_user/<int:user_id> [POST]: Enforces current_user restrictions specifically to drop SQL rows definitively.

## 3. Data Design

### 3.1 Data Model

- User Table: The main database table that saves the user's ID, Username, Email, Password (securely hidden), and the exact time they last logged in.

### 3.2 Data Access Mechanism

- Database Connection: We use Flask-SQLAlchemy so Python can talk directly to the database. This means we write Python code (like User.query) instead of messy SQL code to find and save data.

### 3.3 Data Retention Policies

- Saving Data: User information is saved safely in the energyguard.db file located in the project folder.

- Deleting Data: User data stays there permanently until an Admin manually clicks "Delete User," which erases their record forever.

### 3.4 Data Schema Migration

- Setup: The database tables are automatically generated by Python the very first time the app is run. This ensures that every developer is looking at the exact same database structure.

## 4. Interfaces

- User Interface: The website is built using standard HTML and CSS. It looks clean and works well on any computer screen.
- System Interface: The Flask application connects directly to the local SQLite database file to read and write information quickly.

## 5. State And Session Management

- Remembering Users: The app uses Flask-Login to keep track of who is currently signed in. It gives the user a secure, hidden browser "cookie" locked by our app's Secret Key so that hackers cannot fake a login.

## 6. Caching

- Fast Loading Tools: The user's web browser automatically remembers the style.css file so the website colors and styling load instantly on every page view.
- Live Dashboards: The dashboard data is *never* cached. This ensures the user always sees fresh, up-to-date energy statistics whenever they open the page.

# 7. Non-Functional Requirements

## 7.1 Security Aspects

- Password Hiding: All passwords are mathematically scrambled using "Werkzeug Settings" so that even if the database is leaked, the actual passwords cannot be read.
- Admin Tools: Certain app features (like deleting users) are physically blocked by the code unless the person who logged in explicitly has the username admin.
- Threat Protection: Using SQLAlchemy automatically defends our app from hackers trying to inject malicious SQL code through our login forms.

## 7.2 Performance Aspects

- High Speed: The mock energy calculations are done instantly in the computer's memory, so the dashboard page loads in just a few milliseconds.
- Lightweight: Using a simple local database (SQLite) instead of a massive database server keeps the app extremely fast and easy to run locally on any machine.

# 8. References

- Flask Guide: https://flask.palletsprojects.com/
- SQLAlchemy Info: https://flask-sqlalchemy.palletsprojects.com/